Universidade Lusófona do Porto

Bot Blast

Game Design Document

Alex Nogueira
Guilherme Isidoro
João Miguel

Videojogos e Aplicações Multimédia
2024/2025
Projeto de Game Design 2

Gotta go Blast.

Executive Summary

Title

BotBlast

Genre

3D Speedrun PLatformer

Dev Team

Alex Nogueira - 2D/3D/UI Artist/Animator, Producer
Guilherme Isidoro - Game designer, Level designer, QA Specialist
João Miguel - Programmer, Sound Designer, Game Writer

Product Overview

Bot Blast takes place in a techy testing laboratory, where the player takes on the role of TARA (Testing Automation Robot Assistant), a robot created by the mad scientist known only as "the Doc". TARA must use the BlastPacks, new inventions created by the Doc, to gain speed and close the gaps between jumps in this fast paced, speedrunning platformer.

One Sentence Resume

Run, jump and blast your way through BotBlast's levels as quickly as possible, and try not to fall trying.

Gameplay Synopsis

BotBlast allows the use of BlastPacks to get through the levels, running on walls, jumping around obstacles and eventually reaching the finish goals. The gameplay includes a lot of trial and error, as players attempt to get better times and better medals on each level.

Target Audience

The game allows for a more casual audience, as the player doesn't have to have mastery of the mechanics to finish each level, yet BotBlast's main audience are hardcore, experienced players of any age and gender who will not rest until they beat all the medals.

Platforms

The game is planned to be released for Windows. The gameplay functions better with quick and precise camera movements, the game was made to be played with a keyboard and mouse, though there is controller support.

Unique Selling Points

Combining the rocket jump-like and wallrunning mechanics from known PvP games. such as Quake, Valorant and Titanfall 2, with the fast paced, precise platforming and medal mechanics from games such as Neon White, Cyberhook and Mirror's Edge, BotBlast gathers a few major selling points such as:

BlastPack System:
The Player can use BlastPacks at any point (given they have been refilled by touching the ground) to gain speed, close distances or perform air maneuvers. This allows for exploration regarding different timings, locations and directions in which the player may use the BlastPacks, to better get through the levels.

High Speed Platforming:
There is little limit to how much speed the player can gain, allowing for very fast paced and frenetic gameplay. This allows for exploration on different possible shortcuts through the levels.

Competitive Replayability:
A time-based medal system consisting of Bronze, Silver, Gold and Author medals works as an incentive for players to replay levels and try to better their times. The Top 5 Leaderboard on each level also allows for competition between players.

Marketing

The game will have a trailer, and be promoted on social platforms such as TikTok, Instagram and Youtube, focusing on short videos that demonstrate the fast paced gameplay and visual aesthetic of the game.

Gameplay

Mechanics

Base Movement

The player accelerates for about a second before reaching their maximum base speed while running. This speed can be surpassed through the use of blast packs and consecutive Wallruns.

BlastPack

The central mechanic of the game. Pressing RMB, Q or E throws out a blast pack in a set trajectory in the direction of where the camera is facing. The blast pack continues to travel until it's exploded by the player or sticks to a surface.
Its travel speed can be affected by the player's momentum and the blast pack will never explode on its own.
Pressing the same button again explodes the blast pack, activating its impulse radius that launches the player away from the blast pack. The launch angle and speed depend on the player's location in relation to the blast pack as well as their own trajectory angle and momentum.
The blast's impulse is omni-directional with no forced trajectories or limitations, opening up endless platforming possibilities. The player can have up to 2 available uses of the blast pack at once.
Only one blast pack may be deployed at once, another cannot be thrown before the currently deployed one explodes due to both actions sharing the same button. Both uses of the blast pack refresh anytime the player lands on the floor and one use refreshes if the player has 0 uses left and is in the middle of a Wallrun.

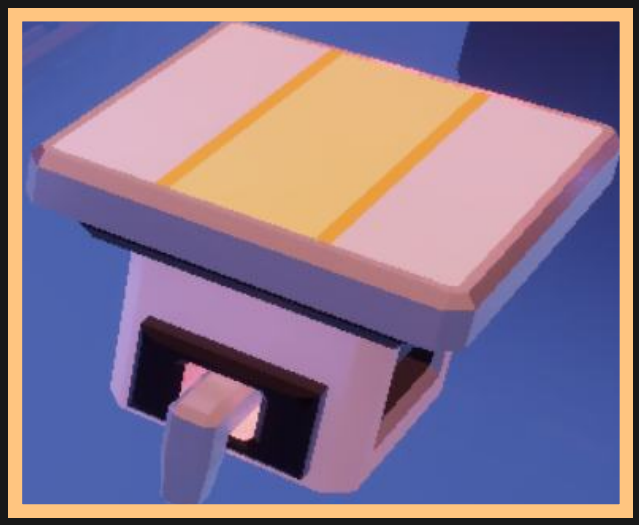
Wallrunning

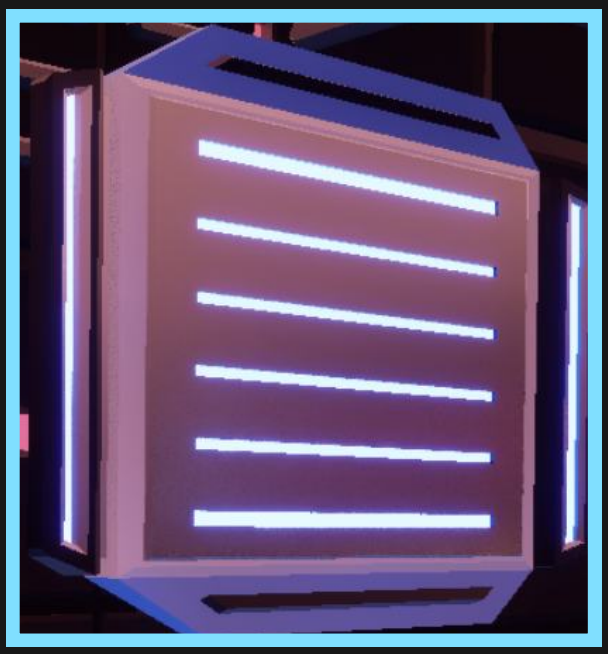
The secondary mechanic of the game. With sufficient speed and a proper trajectory angle, the player is able to attach to Wallrun Blocks and run directly forward along its length.
The player detaches manually by pressing Space to jump off of the Wallrun Block, or automatically if they reach the end of the Wallrun Block's length or turn the camera to be at an angle where they're no longer able to remain attached.
Once the Wallrun is initiated, the player is locked going forward and cannot alter their direction without detaching.
As previously stated, the Wallrun refreshes a use of the blast pack if the player has none left.

Level Elements

Platform Blocks

Floating blocks of solid ground the player is able to stand on and jump to that allows them to reach the goal, making up the foundation of every level. They refresh both blast pack uses upon landing on them from a jump.
There are 3 sizes of platform blocks: Small, Medium and Large. Each size also has a variant with a light on it, placed for specific sections where the player is encouraged to use at least one blast pack to reach the next platform.



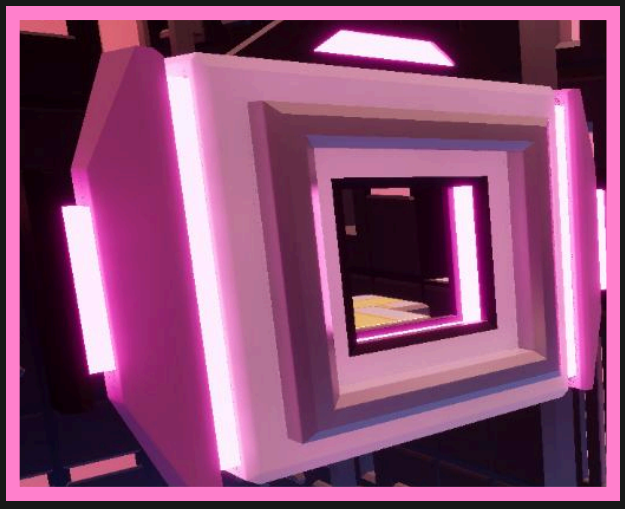


Wallrun Blocks

Blue walls that the player is able to wallrun on. Making contact with the wall to initiate a wallrun refreshes one blast pack use if the player has none left.
There are 3 sizes of wallrun blocks: Small, Medium and Large.

Stop Blocks

Red obstacles that the player can't stand or wallrun on, needing to be jumped around or circumvented in midair.
There are 2 sizes of stop blocks: Small and Large.



Donut Blocks

Donut shaped obstacles that have an opening in the center for the player to jump through.
The player is also able to stand on them, both on the solid ground of the opening in the center or on top of the block.
Like platform blocks, landing on them refreshes both blast pack uses.
There is only 1 size of donut block.

Game Loop

Upon starting the game, the player can choose to play the Training levels or any of the Test Chamber levels, all of which are unlocked from the start.

If the player chooses to follow the level order, they would start with the Tutorial level which is designed to teach them the controls, the mechanics of the game and provide them with easy-to-practice platforming sections thanks to the lack of a lethal pit like in regular levels and auxiliary ramps that aid in traversing the level and getting back up after missing a jump.
Upon completing the Tutorial, the player would decide between practicing further in the Training Grounds or proceeding to Chamber 1.

In the Chamber levels themselves, the player must reach the goal in order to complete the level. They're able to restart at any moment in order to re-attempt before their first completion of the level if they so wish, but upon completing the level they are awarded a medal based on the timer.
The player can then move on to the next level should they be satisfied, or decide to restart in order to chase a better completion time and achieve a more difficult medal.
The player may also decide to aim for the optional Collectible, searching the level structure to find it and then discover a way to collect it that allows them to reach the goal afterwards. Doing so would grant them special dialogue that expands on the game's story.

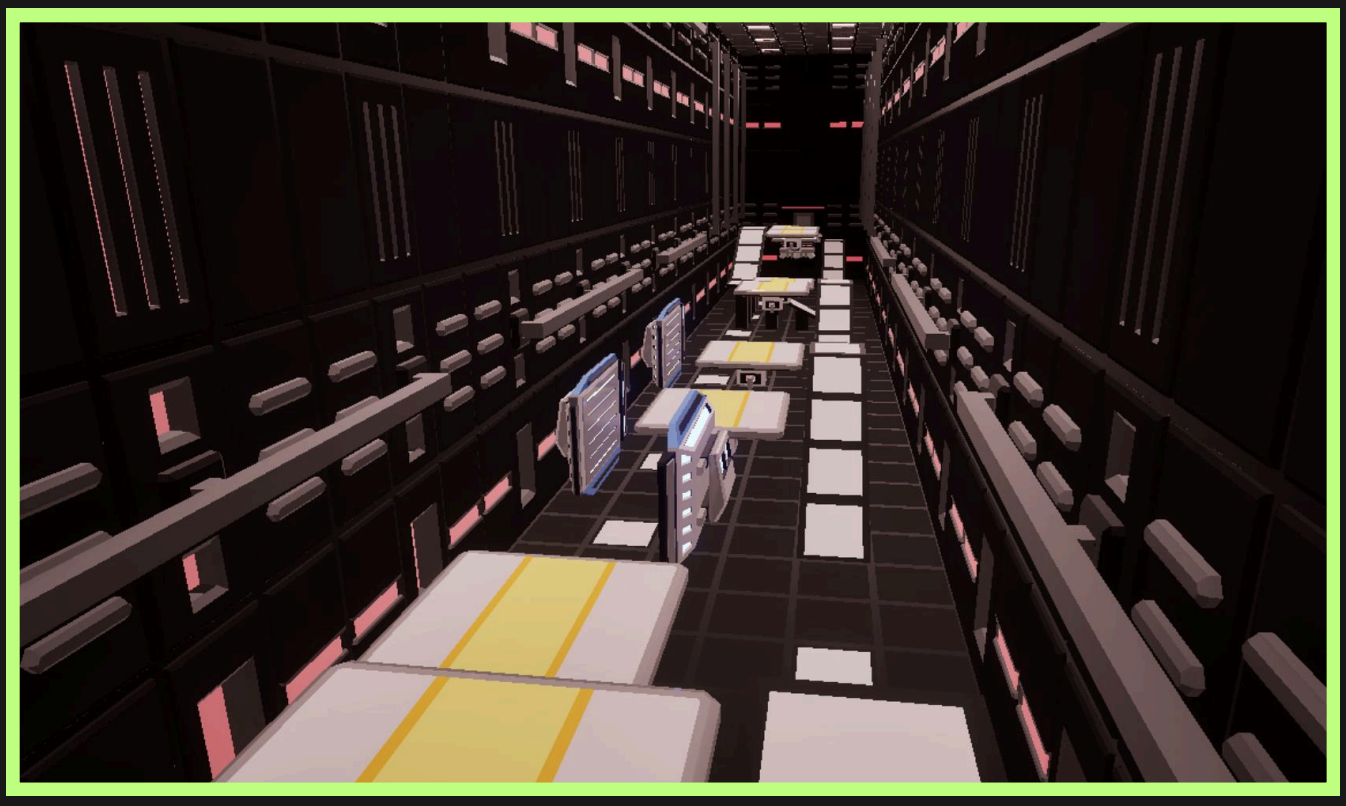
The player is free to attempt any level they want as many times as they'd like, encouraged to do so due to the medal system and the time for the next achievable medal displayed in the results screen.

Level Design

There are 2 training levels and 9 traditional levels named Chambers. The 9 Chambers are split between 3 Tests, grouped together based on their difficulty.

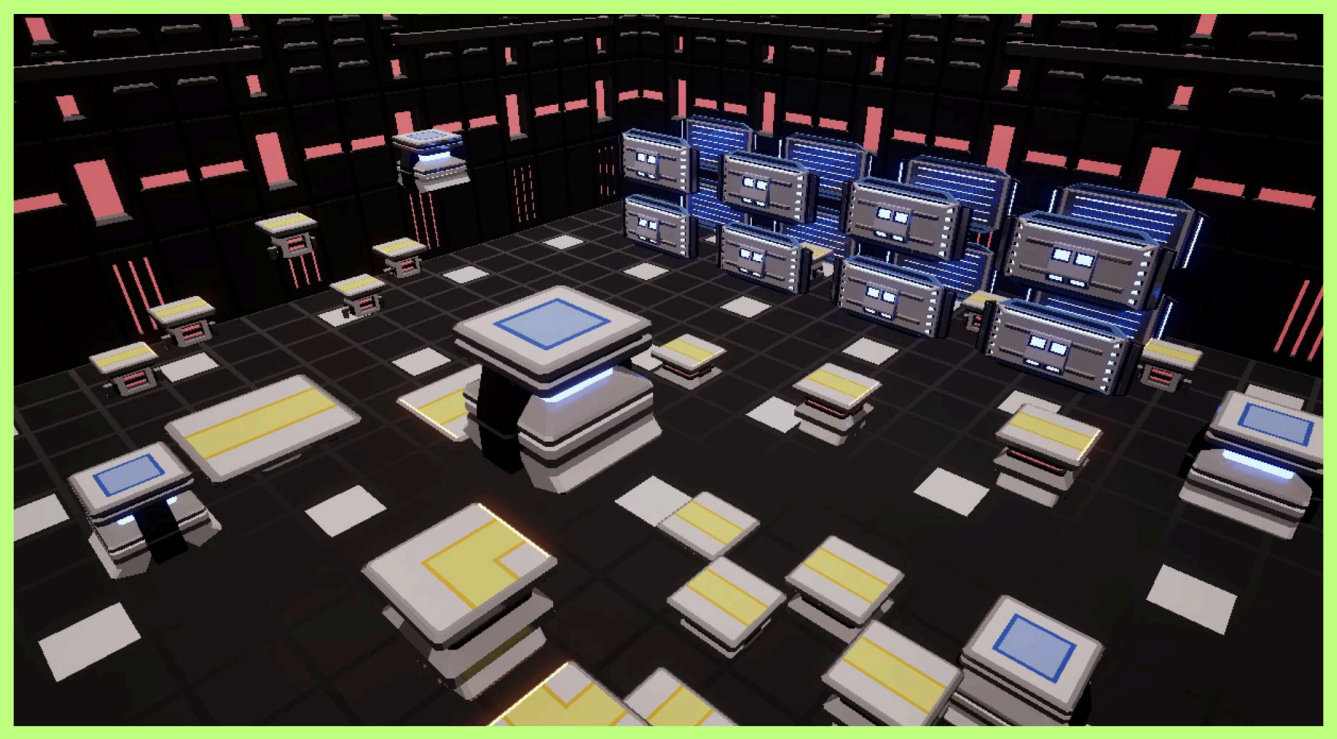
Tutorial

A level meant to introduce the player to the game's movement, mechanics and platforming structure.
It's a linear level with various jumps placed in a straight line, guided by the Doc as he progressively explains the game's controls. Unlike traditional Chamber levels, there's a safety net in the form of a floor and ramps leading back up to the jumps that allow for a more forgiving, easygoing experience.
However, despite being a tutorial, some later jumps can prove a little tricky for beginners who are still growing accustomed to the Blast Packs, further solidifying the convenience of the ramps as they allow players to retry the later jumps without needing to restart from the beginning.



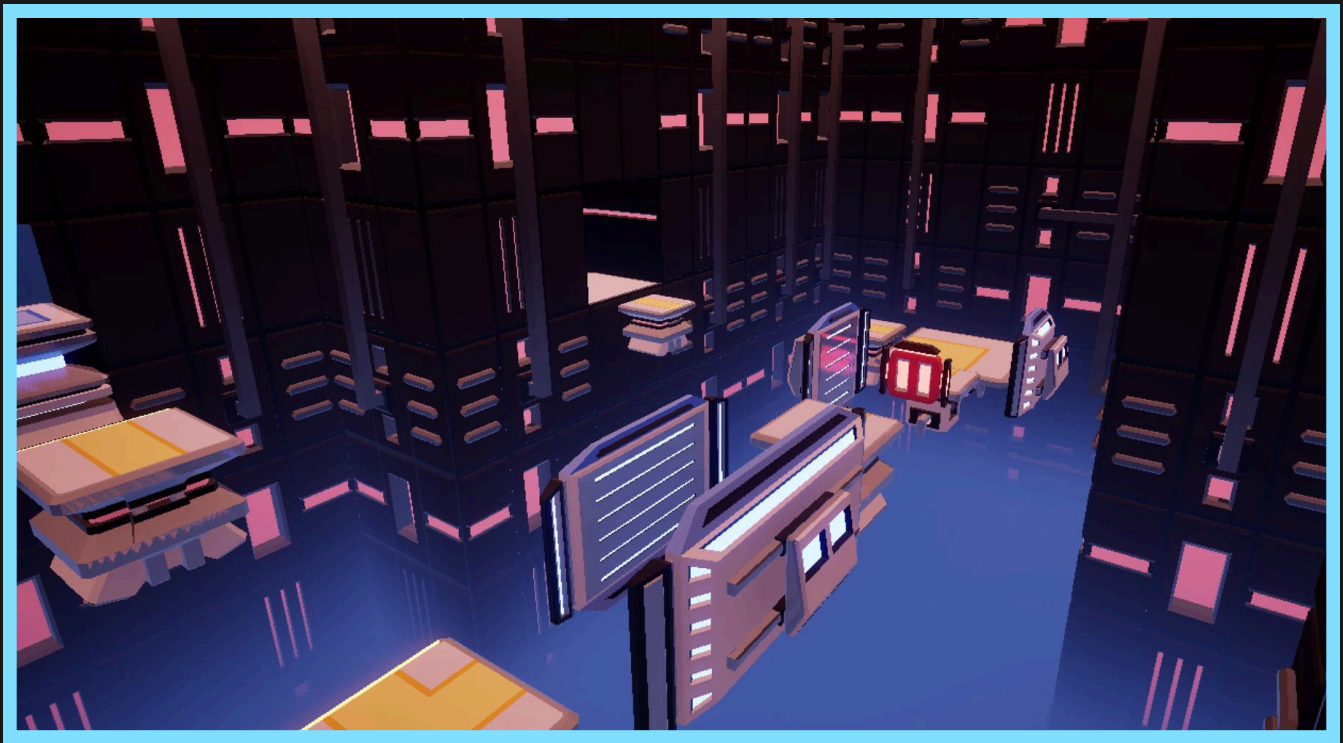
Training Grounds

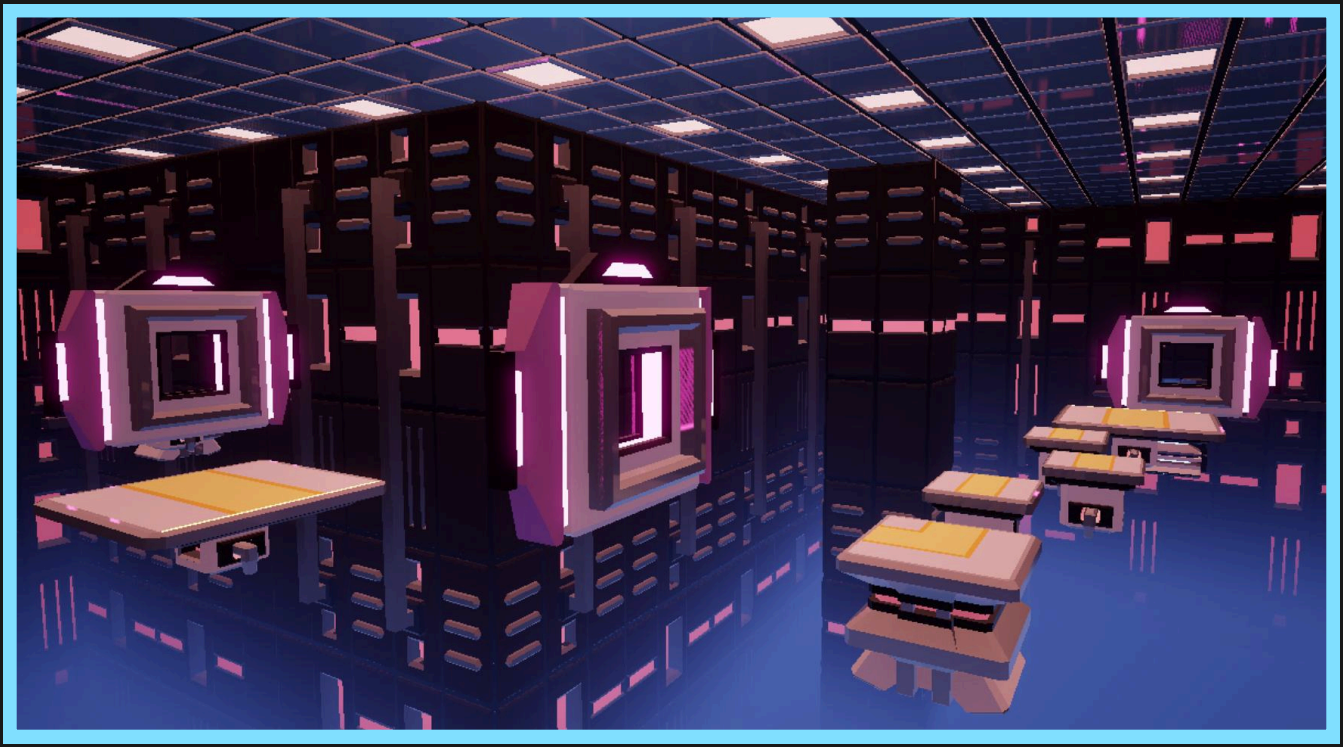
An open recreational area designed to allow players to freely platform and practice their use of the Blast Packs and Wallruns with no goal or timer to worry about. Just like the Tutorial, there's a floor spreading across the whole room that keeps the player safe from any deaths.



Chamber 1

The first traditional level, introducing the player to the level structure as they navigate short gaps and the game's most common obstacles to reach the goal.
The platforms are placed close to each other, offering the player a simpler, first challenge to build confidence. Despite being the first level, there's still plenty of room for optimization and shortcuts, with most initial platforms presented at the beginning of the level being totally optional with clever use of the Blast Packs.
The potential for replayability and improvement becomes immediately apparent as even in the very first level, earning a Gold medal proves to be quite the challenge.



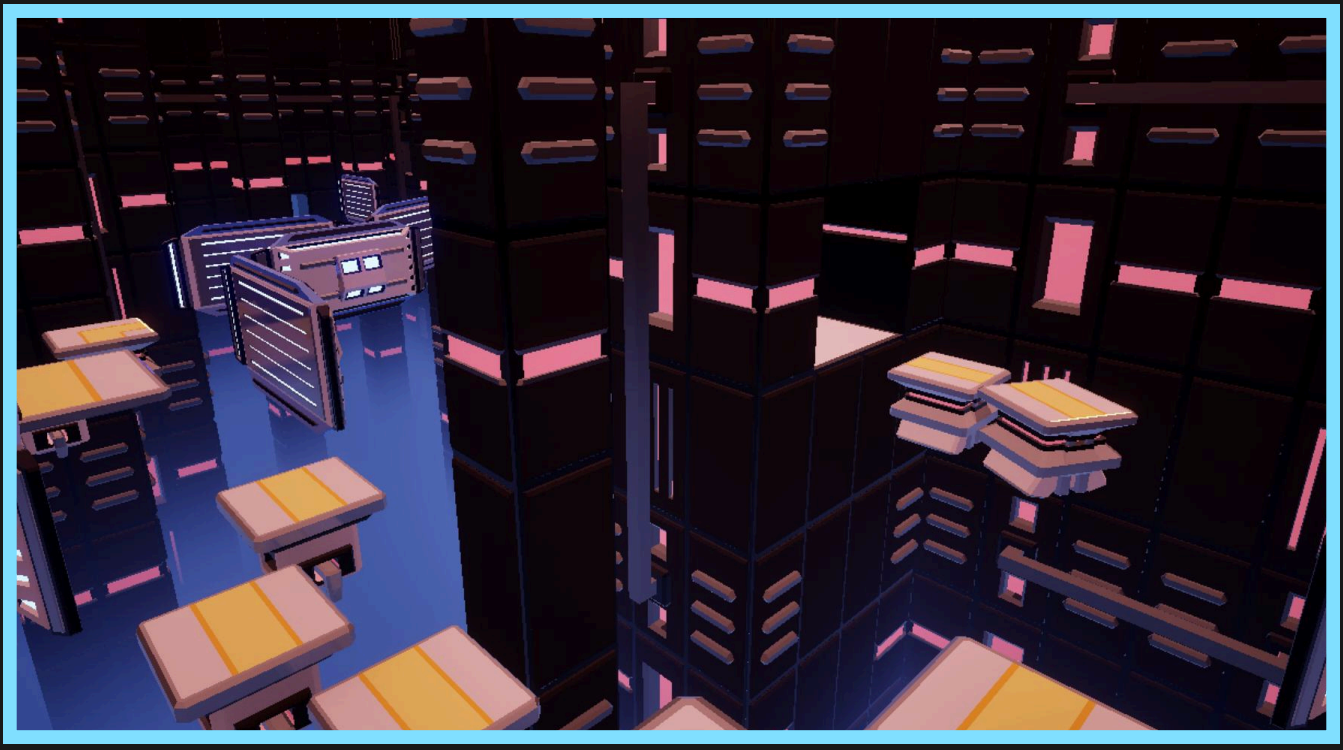


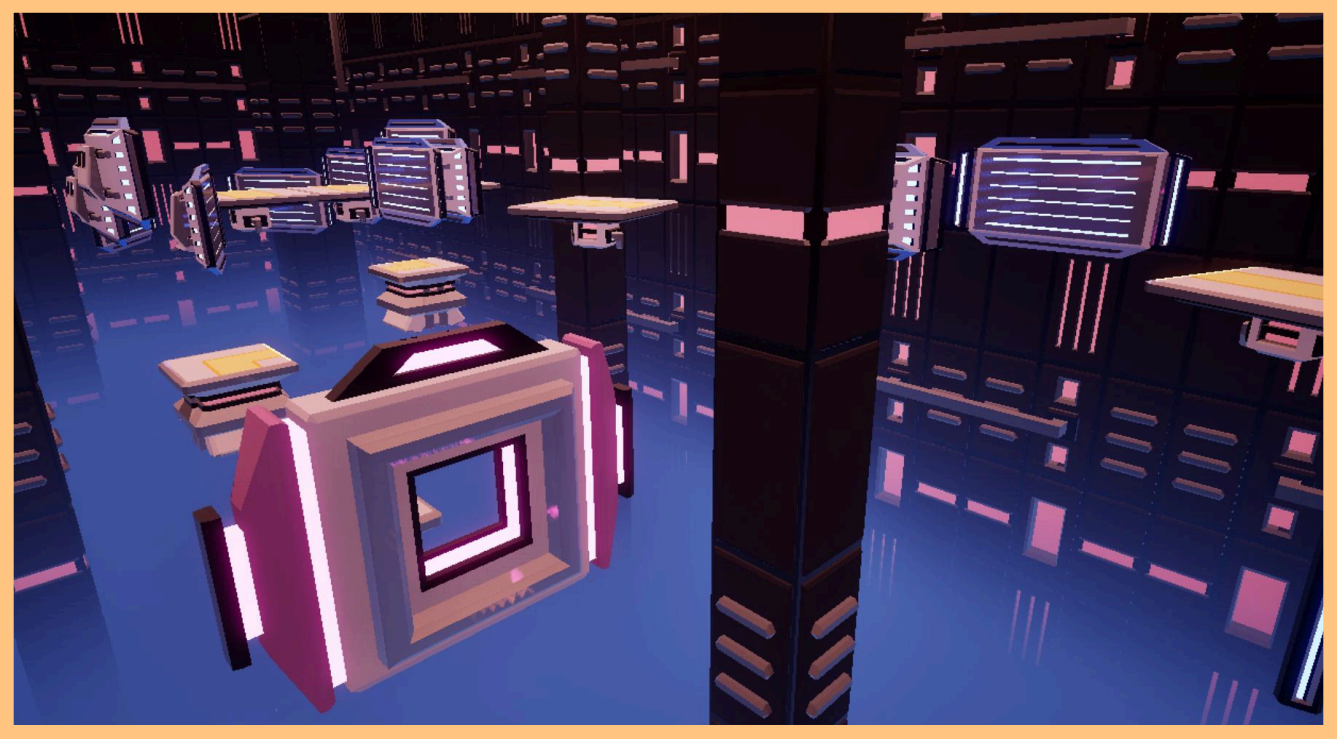
Chamber 2

The second level of the first Test. The platforms are still placed in a forgiving, beginner-friendly way, but the introduction of the Donut blocks challenges the player's adaptability and thoughtful use of their Blast Packs.
The jumps through the Donut blocks can be precise, especially the last jump that leads straight into the goal, but nothing intimidating due to the relatively low difficulty and short level structure that allows for plenty of attempts.
Once again, there's also potential for optimization with possible gap jumps that skip multiple platforms.

Chamber 3

The final level of the first Test. Its level structure is longer and slightly less linear than the ones before it, but follows a similar logic with a higher emphasis on Wallruns.
It introduces the player to consecutive Wallrun blocks that require the player to jump from one Wallrun into another, a skill that's tested in later, harder levels.
Out of the first 3 levels, this one has the highest potential for long shortcuts, as depending on the player's approach and routing a majority of platform blocks or Wall blocks can merely be considered a suggestion.



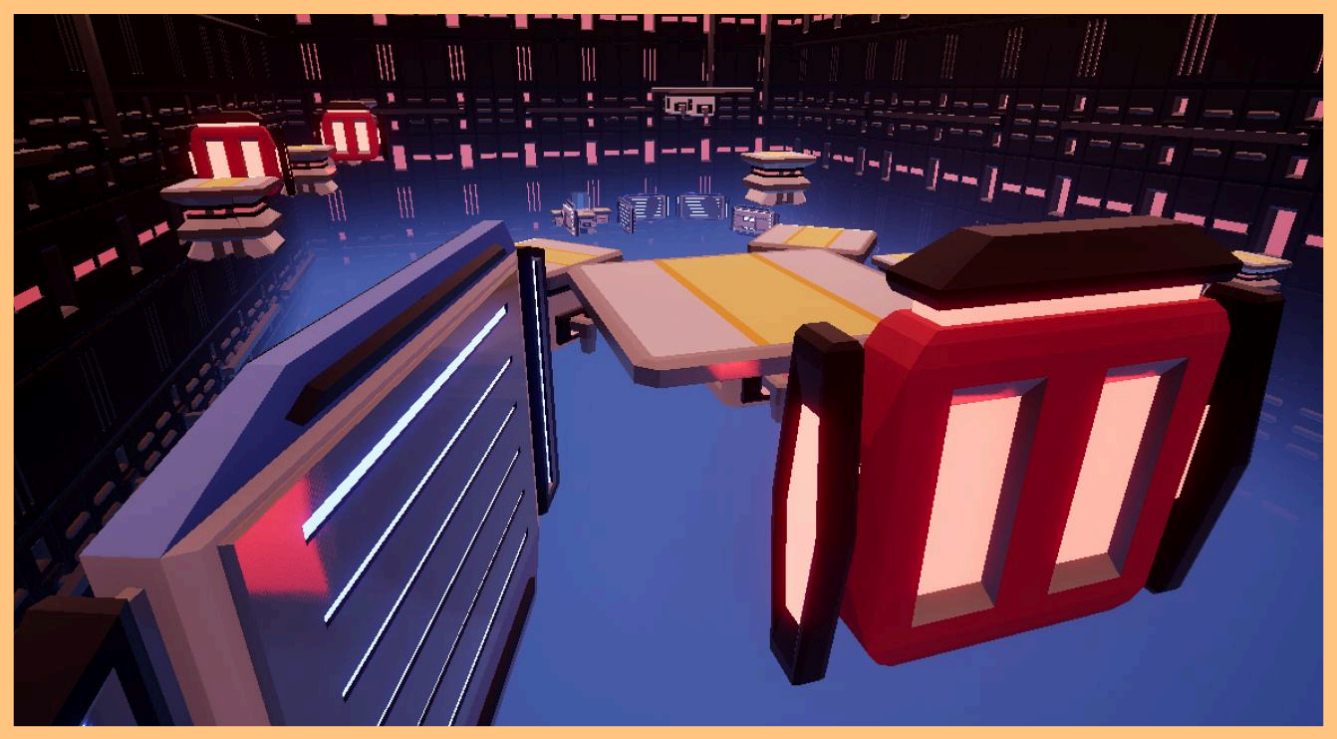


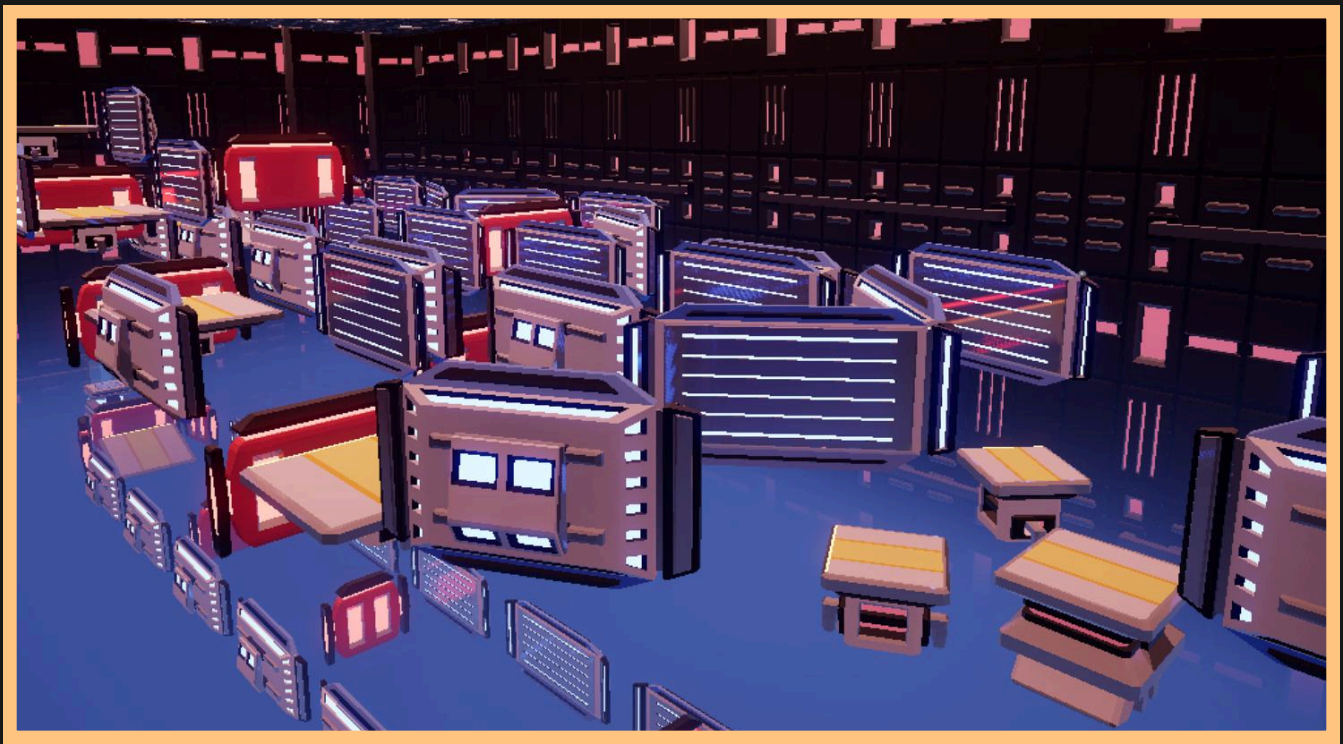
Chamber 4

The first level of the second Test that introduces the player to the common theme of the upcoming levels: multiple available paths. Halfway into the level, there's a fork in the road that offers the player two distinct paths with their own unique platforming.
The right path features long, consecutive Wallruns while the left path has a Donut block jump and gaps that emphasize the use of the Blast Packs over the Wallrun.
The paths then converge at the end leading to a large pit that can't be cleared through the use of Blast Packs alone, requiring long, consecutive Wallruns to reach the goal.
There are also additional Wallrun blocks present at the start of the level, an optional challenge that leads to the Collectible and allows the player to reach high speeds if they clear the tricky jumps between the Wallrun blocks.

Chamber 5

The second level of the second Test, expanding on the concept of multiple available paths by presenting the player with multiple forks. The level splits into two paths at the very beginning before another subsequent split, leaving the player with 3 distinct routes that all lead to the goal.
The difficulty starts ramping up at this stage of the game, as this level's structure focuses on the use of both Blast Packs to clear larger gaps.
The increased challenge is notably present in a particular section that requires the impulse of two Blast Packs into a Wallrun and another section where the player must strafe to avoid Stop blocks in midair.
The varied obstacles and multiple paths test the player's decision making and offer even higher replayability. It's also notable that all paths are almost equally viable for reaching the harder medals, depending on the routing and the use of Blast Packs.



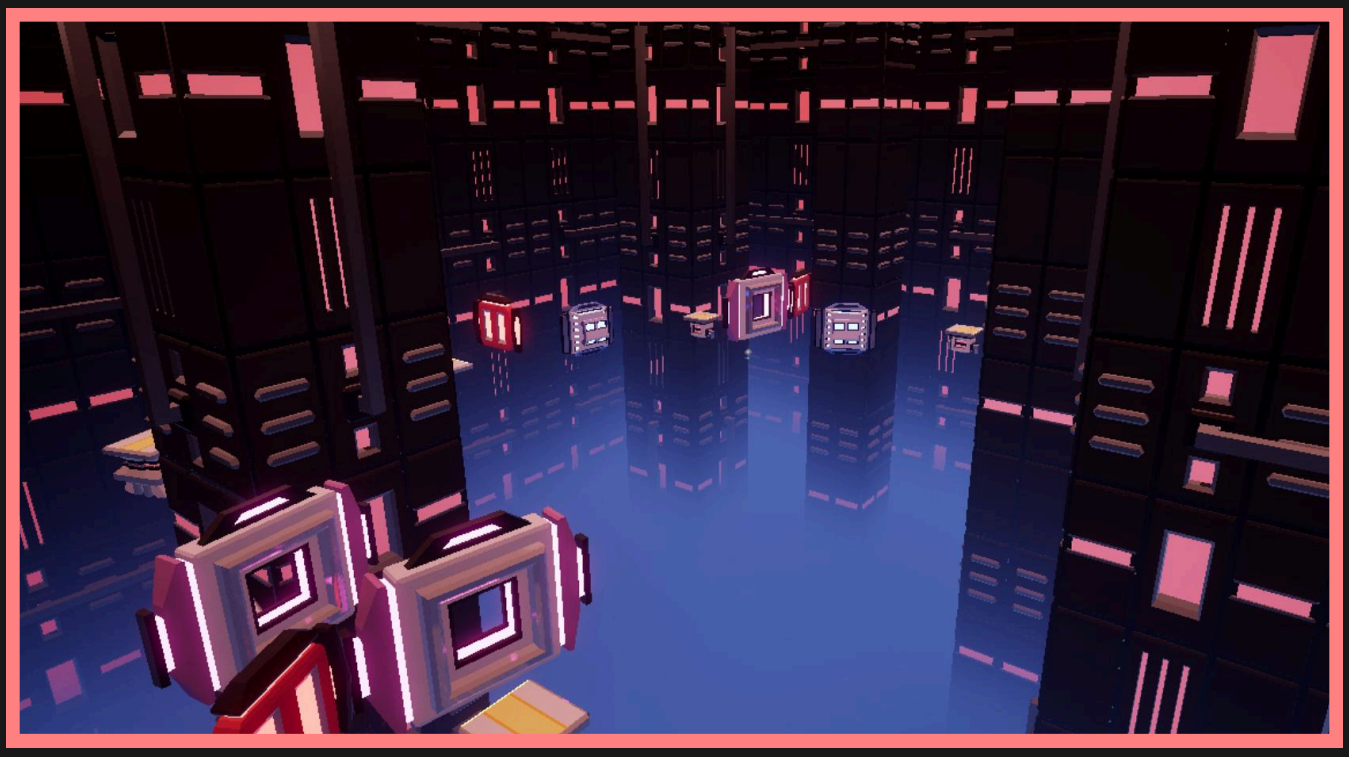


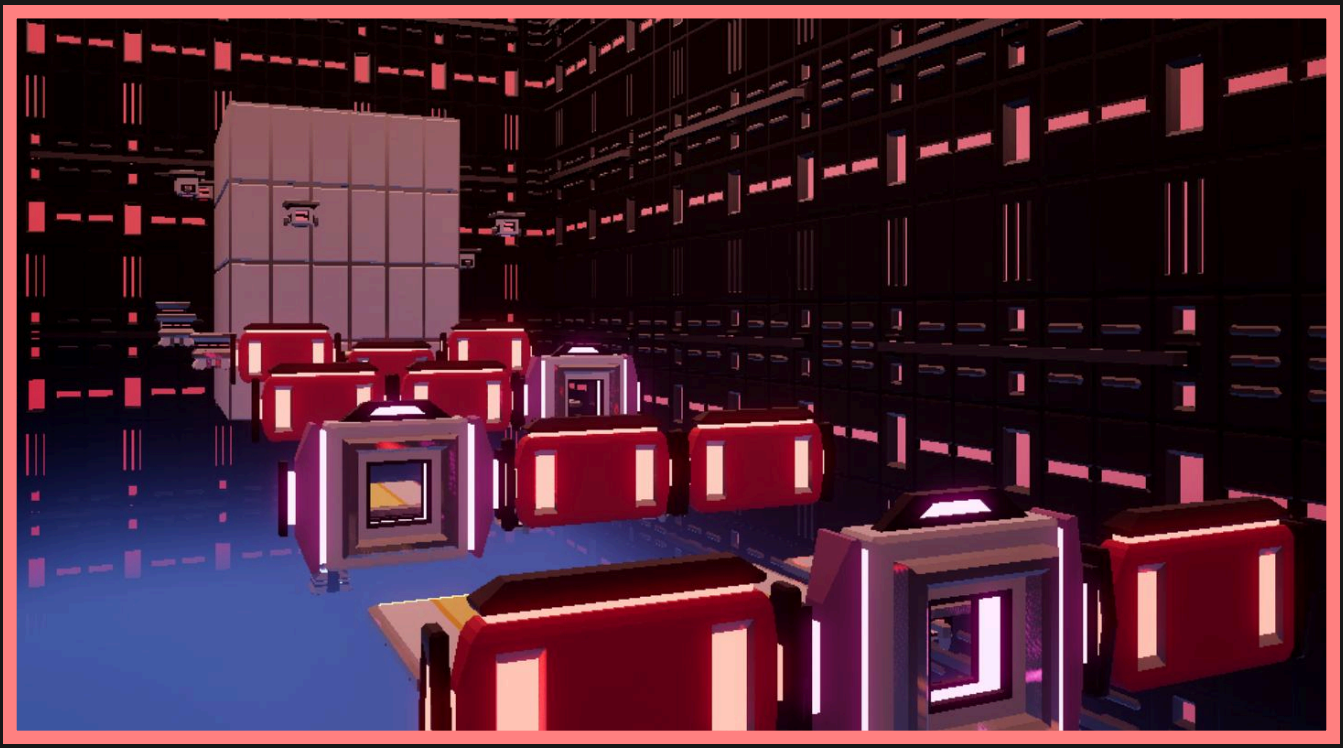
Chamber 6

The final level of the second Test, an appropriately challenging Wallrun-focused level designed to prepare players for the difficulty of the upcoming third Test.
Once again, forks and multiple paths are presented to the player where they can pick their poison on which kind of platforming challenge they'd rather tackle.
The whole level primarily focuses on Wallruns, but there's still variation in the differing paths as players can pick between tricky consecutive Wallruns that turn a corner or precise, tall jumps that require the use of Blast Packs that launch upwards towards Wallrun blocks that are otherwise out of reach.
There's an additional path present at the very bottom of the level, a notably challenging corridor of consecutive Wallruns obstructed by Stop blocks available to the more brave, challenge-hungry players that upon completion usually secures the time for a Gold medal.

Chamber 7

The first level of the challenging third Test.
The primary focus of the level's design is precision, presenting the player with difficult jumps they've never seen before that require thoughtful planning on how to strafe in order to avoid obstacles and reach the distant Wallrun blocks.
It's a relatively linear level with less potential for large shortcuts, the available optimization being mostly centered around the speed at which players can clear the precise jumps and memorize the layout to their advantage.



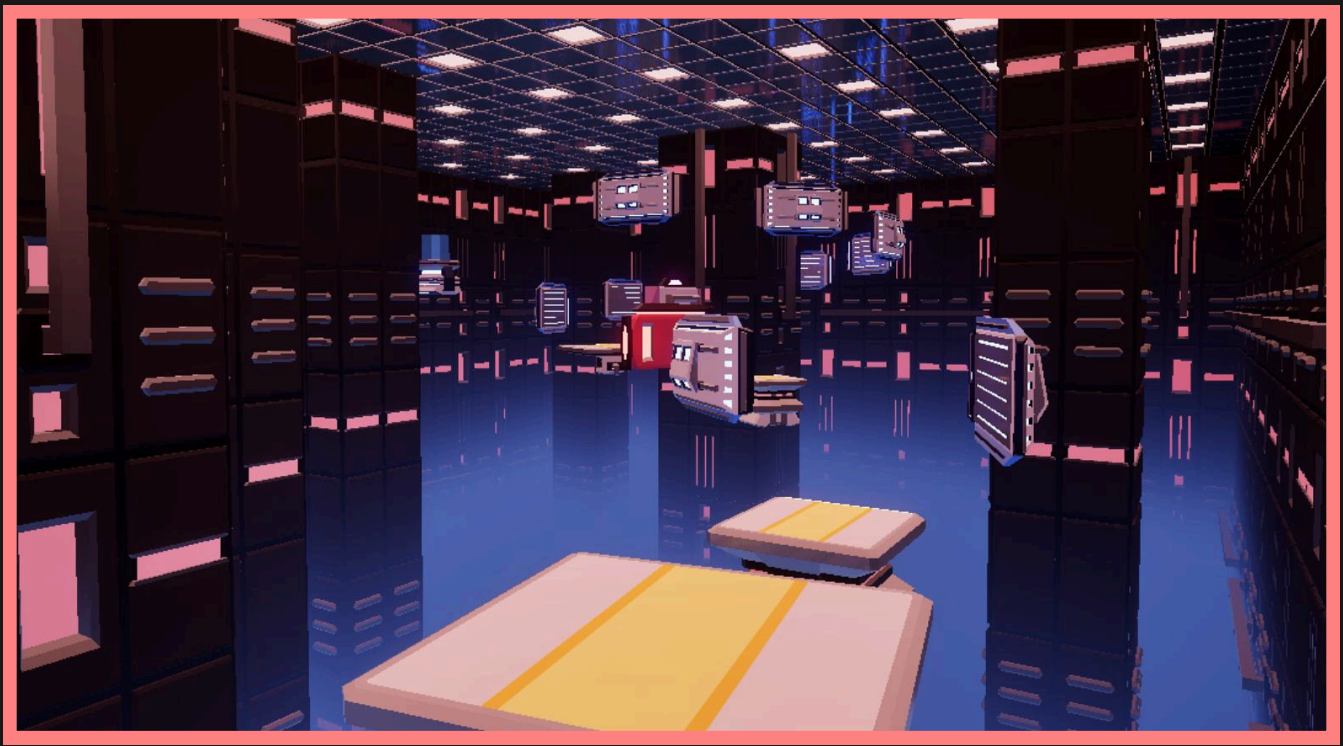


Chamber 8

The second level of the third Test and the longest level following its intended path. The player's planning and practiced, mechanical use of the Blast Packs is thoroughly tested as they navigate the precise Donut block jumps and are greeted with a unique vertical platforming section that spirals upwards to the top.
The end of the level offers a unique "dropper" experience where the player falls from the top of the tall structure, avoiding platforms before landing straight onto the goal. The player's patience is challenged by the long, vertical section.
However, the player's innovation and creativity is also secretly challenged, as despite the intended route being the game's longest it's actually possible to skip it entirely with one, very precise jump from an earlier section straight towards the goal.
That's the game's largest shortcut, hidden in plain sight for the smartest of players to uncover.

Chamber 9

The final level of the game, combining precision with high speed as the player's presented with some of the game's toughest sections.
In particular, the initial section where the player must Wallrun and jump through Donut blocks with enough speed to reach the next Wallrun, as well as the final section with a spiral of consecutive Wallruns that send the player flying into the goal will surely cause many restarts.
Above all else, the player's mechanical skill at using the Blast Packs and adaptability is tested in this level as they balance speed with control in midair throughout multiple runs.
It's the most difficult level of the game, befitting the final challenge.



Medals

The player is rewarded with a medal after a run through a Chamber based on the time taken to reach the goal.
There are 4 medals: Bronze, Silver, Gold and Author. Each medal is harder to obtain than the last, requiring faster runs with more precise jumps and clever shortcuts.

The times necessary for each Chamber's medals were chosen with the following balance in mind:
Bronze medals are relatively easy to obtain with leniency for mistakes or slow jumps.
Silver medals are harder to achieve but not too challenging, requiring a good pace through the level's intended route.
Gold medals are notably difficult, forcing the player to navigate the level optimally and think outside the box with shortcuts.
Author time medals are the fastest times achieved by the developers, true challenges only for the most determined of players that push the level's design and the game's mechanics to their limit.

The times of all the medals for each level are as follows:

	Bronze	Silver	Gold	Author
Chamber 1	20.00	15.00	10.00	6.09
Chamber 2	20.00	16.00	12.00	8.18
Chamber 3	27.00	22.00	17.00	8.13
Chamber 4	36.00	28.00	20.00	10.87
Chamber 5	32.00	27.00	22.00	16.51
Chamber 6	33.00	27.00	21.00	8.85
Chamber 7	42.00	37.00	32.00	27.80
Chamber 8	1:07.00	57.00	47.00	12.43
Chamber 9	40.00	33.00	26.00	19.21

If the player has previously achieved a medal, it remains obtained, permanently saved and on display during subsequent runs.

Collectibles

There is one Collectible per Chamber, positioned in hidden or hard to reach places within the level.

To obtain a Collectible, the player must collect them during a run of a Chamber and then reach the goal.

They serve as an extra, optional objective that adds further replayability and rewards the player with hints about the game's story.

Art

The art of BotBlast is rather simple, focusing on interesting lighting and smooth surfaces. The 3D art is low poly, and rather simplified, with a limited number of materials, and very few textures. The 2D art is similarly simple, with a reduced color palette and little detail.

Interface

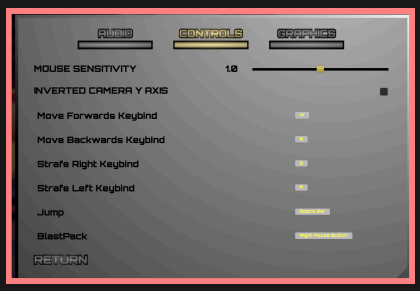
BotBlast's interface is fairly simple. It has a techy, metallic feel, with lit up buttons that react to mouse interactions. The font used, Orbitron, also works towards the futuristic feel.
Hovering buttons makes them light up, and clicking them gives them a color. Buttons are color coded.
Blue relates to playing; Green relates to repeating a level, and training; Yellow relates to the settings and medium difficulty levels; And red relates to hard levels and quitting the game.

Main Menu

The Menu has three main buttons, Play, Settings and Quit, and the game Logo, layered over a splash art of Tara wallrunning.

Clicking the Play button will lead the player to a window with four tabs, one for each test, along with the training tab. Clicking any test tab reveals the three chambers in that test, allowing for the player to select which level to play, depending on difficulty.
Similarly, clicking the training tab will lead the player to the tutorial, and the training chamber. Each tab also displays the medals acquired in each chamber, as well as a picture of each chamber, with the record time overlaying it, and a leaderboard for that chamber below.

Clicking the Settings button will lead the player to a similar window, with three tabs, one for each category of settings, allowing the player to better look for the different types of settings they may want to change.



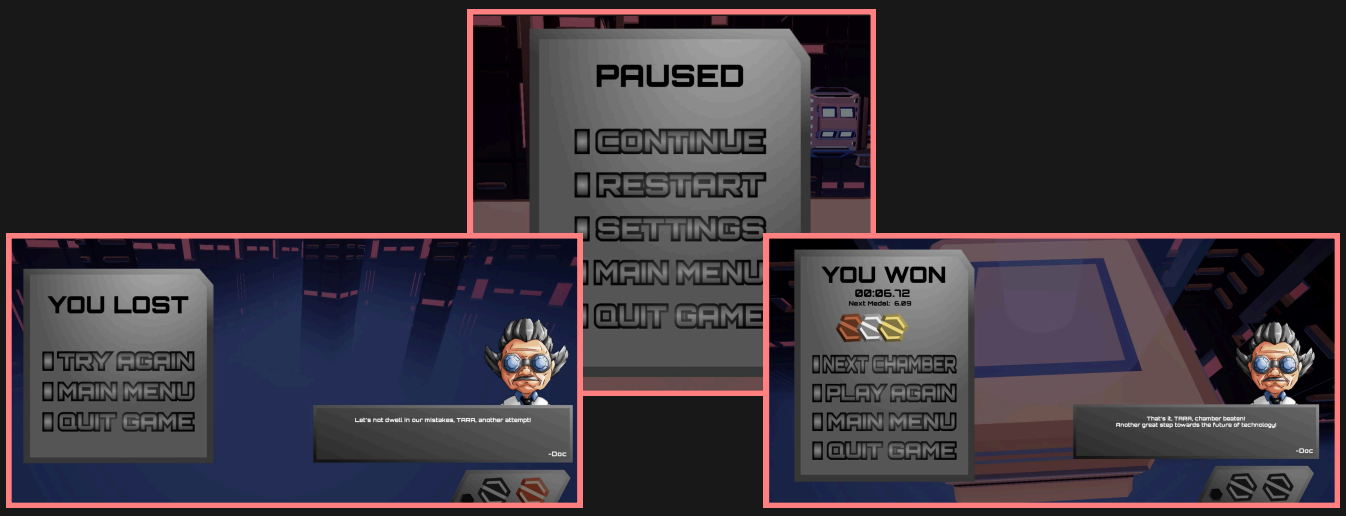
Both windows contain a Return button, to return to the initial Menu page.

Clicking the Quit button quits the game.

InGame

In the game, there is a pause menu, with five buttons, Continue, Restart, Settings, Main Menu and Quit Game.

There are also a win and a death screen, both containing Try Again, Main Menu and Quit buttons, and dialogue from the Doc, and the win screen has the option to go to the next chamber as well.



The player's HUD contains a BlastPack indicator, allowing the player to see the amount of BlastPacks available, and if there is an active one or not. The HUD also contains a timer, allowing the player to see how long it has been since the start of the level.

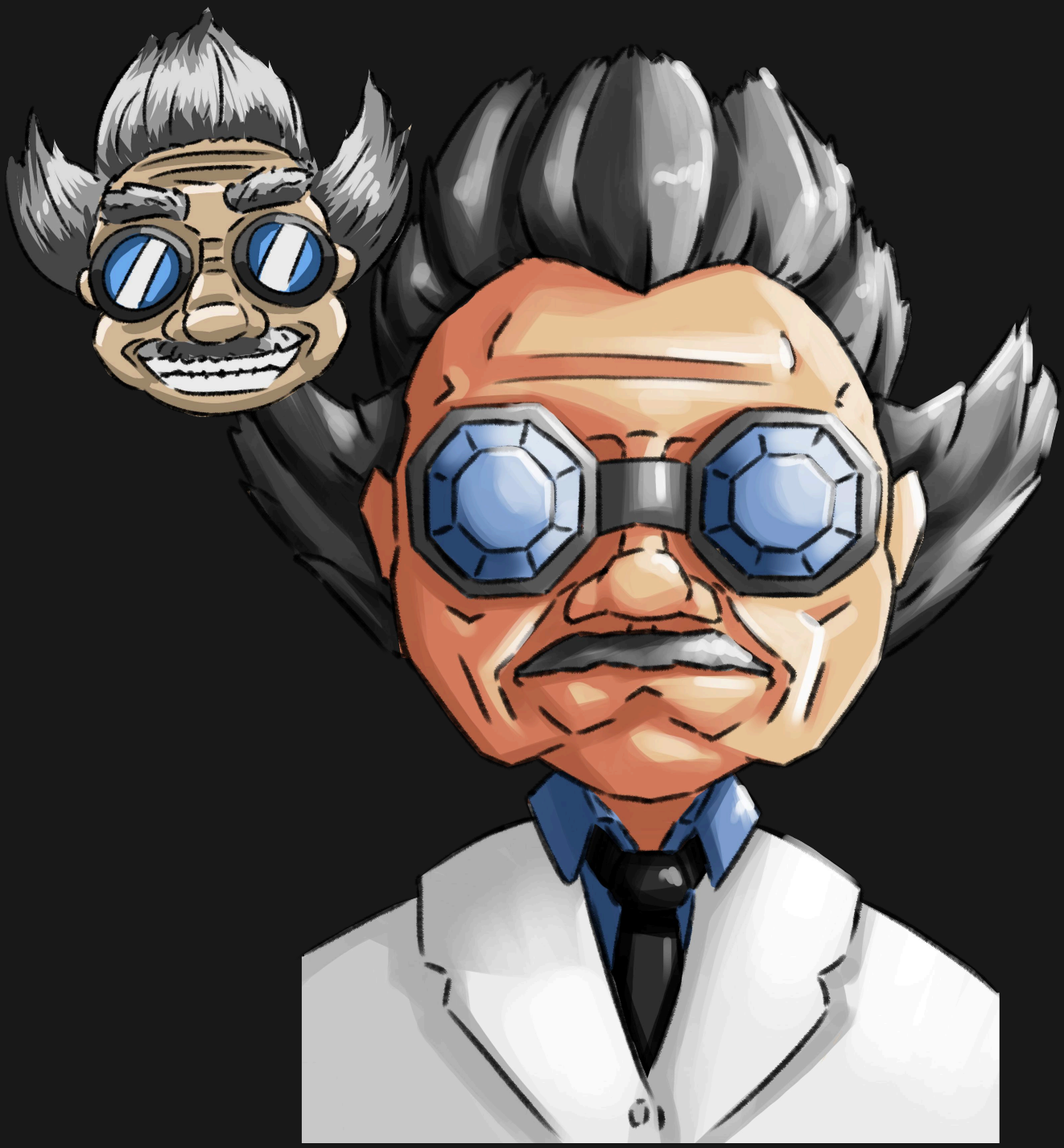
Characters

TARA

TARA's design is rather simple. She is a humanoid robot, with a reduced color palette, and made to fit the low poly style of the game.
Her proportions are fairly cartoony, to better fit a game that doesn't look to be very realistic.
There is a somewhat hourglass shaped pattern on her lower torso as well, planned as a nod to the "timed runs" aspect of the game.
Her colors are red, black and white.
The red, being the stronger, more prominent color, represents TATA's determination and passion, along with her love for her creator and father.
The black represents both her discipline and seriousness to complete her tasks, and unfortunate lack of humanity, being, arguably, a soulless robot.
Lastly, the white represents her purity and innocence, as all she wishes for is to help the Doc, as they develop technologies to better the world.

The Doc

The Doc's design is a classic "Mad Scientist" design. Messy, spiky hair, big, odd glasses, and a lab coat.
His original first sketch featured him smiling widely, but as the game's lore developed to a grieving story, the Doc got a more serious face.
His final artwork is fairly geometric, matching the low poly look of the 3D art, and the geometric 2D UI.



World

Lab

The laboratory was made to have a techy, futuristic feel, with an emphasis on metallic surfaces and interesting lighting.

Each of the different platforms is color coded.

The regular walking platforms are yellow, and have light strips at any ends where the player is meant to use a BlastPack.

The wallrunning platforms are blue, and have horizontal strips to emphasize the speed and movement the player can gain with these platforms.

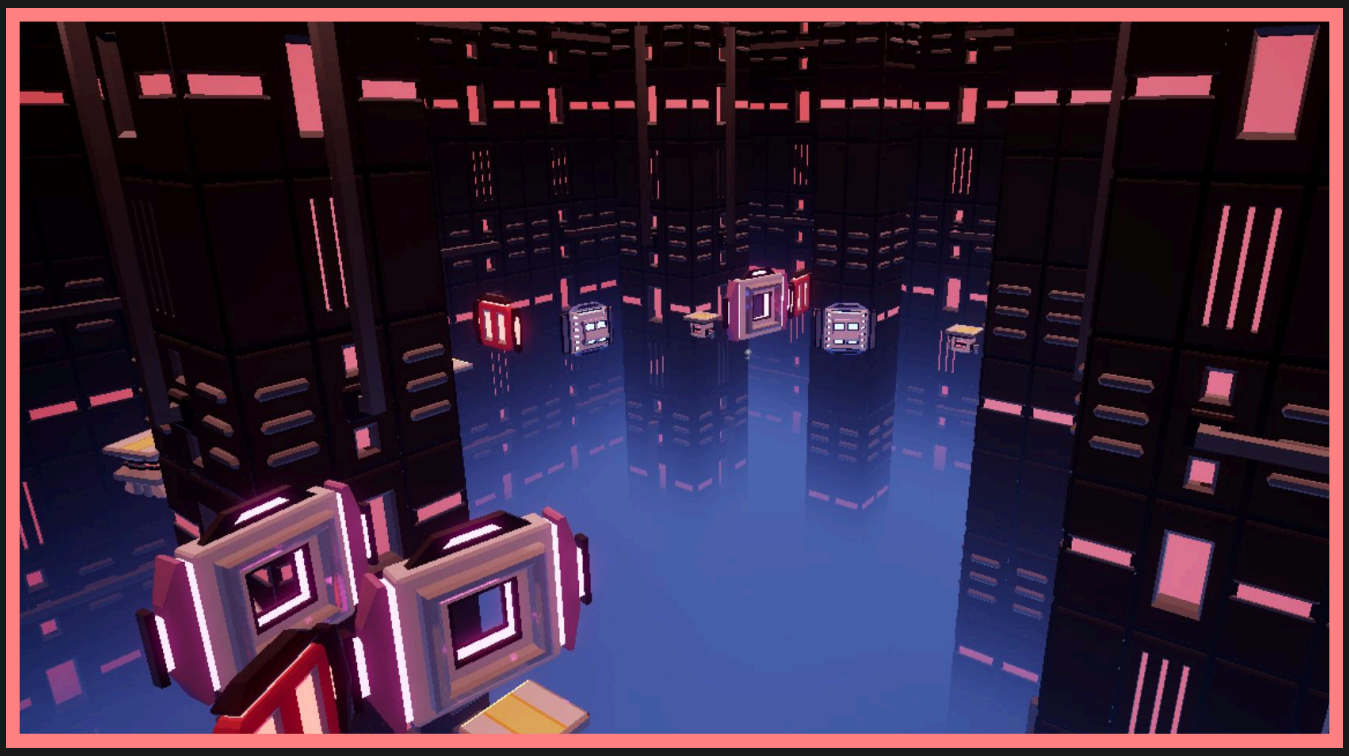
The stopping platforms are red, as they are meant to block and stop the player, being major obstacles.

The donut platforms are pink, simply a contrast color, but still, enough to be recognizable by the player as that specific obstacle.

World

Lab

All the platforms are colored in lighter colors, a way to gain contrast with the darker background environment, ensuring the player is able to tell the platforms apart from the scenario.



Programming

This part of the document serves to follow the production process of BotBlast. It aims to contextualize and document what was done, how it was done and the difficulties and lessons.

The Beginning

In the beggining of the development, one of the main mechanics was already created, the satchel(as called internally, Blast Pack in-game), that was created as a prototype or initial demonstration for the initial pitch of the game.

As such, the initial focus was of creating the player movement, seeing that it is a fast paced game, a good and satisfying movement is crucial. All with the intent of creating a prototype for what the game could be.

We then turned to the wallrun system, that gives the player the ability to run on the walls during a time. This was early on defined to be one of the defining mechanics of the game, and needed immediate implementation.

Using the Blueprints system in Unreal Engine, a small prototype was created, that died shortly due to several problems with the engine, which leads us to:

Unreal Engine

Being the chosen game engine, Unreal Engine is extremely versatile and powerful, but complex. This complexity, coupled with our inexperience with it, let to certain problems.

From the beginning the idea was to create the game using the c++ format in Unreal, that is quite robust but… problematic. The prototype, whilst using only bluprints, was working (more or less) well, but for some reason, the default model and template didn't allow for the wallrun system we were implementing to work well, so we tried to change to c++, and all hell broke loose.

The project stopped working, forcing us to start from scratch, creating a new project and not even being able to access the code created before since it was unreachable inside the broken blueprint.

No big deal, problems happen, we can create a new project, using c++ from scratch to avoid transition problems. But more problems appeared, even in empty projects with hundreds of errors in Visual Studio. After many headaches the solution appeared: abandon Visual Studio.

From this moment onward I (the programmer) used Jetbrain's Rider, an IDE that is much better and more suited to work with Unreal Engine, making the technical development much smoother and with less big errors that slowed development.

Main Mechanic

With the project now working, we returned to implementing the Wallrun.

The wallrun was mostly done through blueprints, since it was one of the first things I did inside a new engine.

So, I used many tutorials for this step. After a few tries and taking a bit from each tutorial, we reached the final iteration of the wallrun quite quickly, even if it did evolve a bit due to game design decisions.

Therefore, in the first two months, the main mechanics were already finalized and with the implementation of the first animations, we had the core concept of the game finished.

The mechanics worked, were fun and jumping around in empty maps was fun, but that is the topic for the game design chapter, so let us move on.

What I do wish to disclose here is the difficulties with said mechanics and one thing that early on I noticed that stayed with me through this project: "We take a lot of things for granted when playing games, and details matter a lot".

The blast packs are very simple, you throw them, they explode and push you forward, as mentioned, that was defined and implemented even before the game existed. But, at first, they were infinite, there was no limit to how many you could use, and flying off the map was an option. Of course, that doesn't make for good gameplay, so it was decided that a limit of two were to be put into place, easy right?

Not so much, limiting it seems easy, for example:

```
Void SatchelCount()
{
            float MaxSatchels = 2

                            if (MaxSatchel>0)
{
                        ThrowSatchel(); [Pretend this function throws the satchel]
                        MaxSatchels -= 1;
}

}
```

If you have a keen eye you must have noticed something, yes, we remove 1 satchel every time it is thrown, if there even is one, but we never add it back. Ok, so do we add it whenever it runs out?

When we touch the ground? Or in a cooldown? Granted, these are all game design questions and not programming, but once decided they had to be implemented, and there is where the edge cases are a pain.

We decided to give back both satchels when landing on the ground (if empty) and one on the wallruns.

Easy, make a check on the tick (that is called every frame) to always give back the satchel if on the ground: Ok, but if I then throw on the ground and jump I now have access to three satchels instead of three: Ok now it only returns the satchel when the active one is exploded, one satchel on the wallrun and fills the satchels when touching the ground, all covered… right?

Once more, no, with some testing we realized that throwing a satchel on the air and landing gave you two satchels plus the one on the world, there was a missing check on the landing, to see if a satchel was active. It ended up staying on the game, since it seemed fun, but it showed how sometimes edge cases can slip through the cracks and how detail oriented one must be. This topic will reappear when we talk about UI/menus later.

Main Mechanic

We spoke previously about how the project was made with the c++ project base but there was heavy use of blueprints, this trend continued throughout the project.

In very broad terms, c++ is more malleable, gives you more tools and possibilities with the drawback of being more difficult to organize/learn.

Whereas blueprints are much simpler and more intuitive but also more rigid. As such, I figured early that using a mix of both was ideal, for example, the player movement was created entirely through c++, but implemented in blueprints with other functions to simplify the addition to the game.

It made for a very interesting process, a tough learning process, but fun. What I really want to make clear in this chapter is the necessity to work with others and understand other people's abilities.

The dev group for BotBlast consists of three members, me, the programmer, an artist and a game designer, which means that apart from me, the other two members are not as versed, or interested in programming and working with code. But are more open to blueprints, seeing that, as mentioned, they are more intuitive and easier to pick up.

That, combined with the fact that we had finished the core concept of the game fast, meant that I had to try and write code that was easily accessible, so when needed, my colleagues could access and use it, and that when we eventually wished to scale the game, we could do so.

This is where blueprintable code came in. Not to get very technical, it just means that whatever function, property, variable or whatever, although created through code in c++, is usable in blueprints. This was very useful to not only allow my colleagues to access theses functions but also work within Unreal's editor and in a faster and more fluid way.

The first truly fully hybrid code I wrote was with the in-game HUD, since the satchels were implemented in blueprints, but the player was in c++. As such, I was able to mend both worlds and create a very robust system. And at one point, the artist wanted to add some more assets and functionality to this HUD he could do without having to ask me to do so and stopping any progress I had made.

Menus, UIs and Systems

BotBlast is not what one would think of when talking about "Menu games", games where the menu or UI is most of the game, for example point and click games, management games, etc. However, it is safe to say that of all the time spent on the programming of this game, over 70% was on menus UIs and systems.

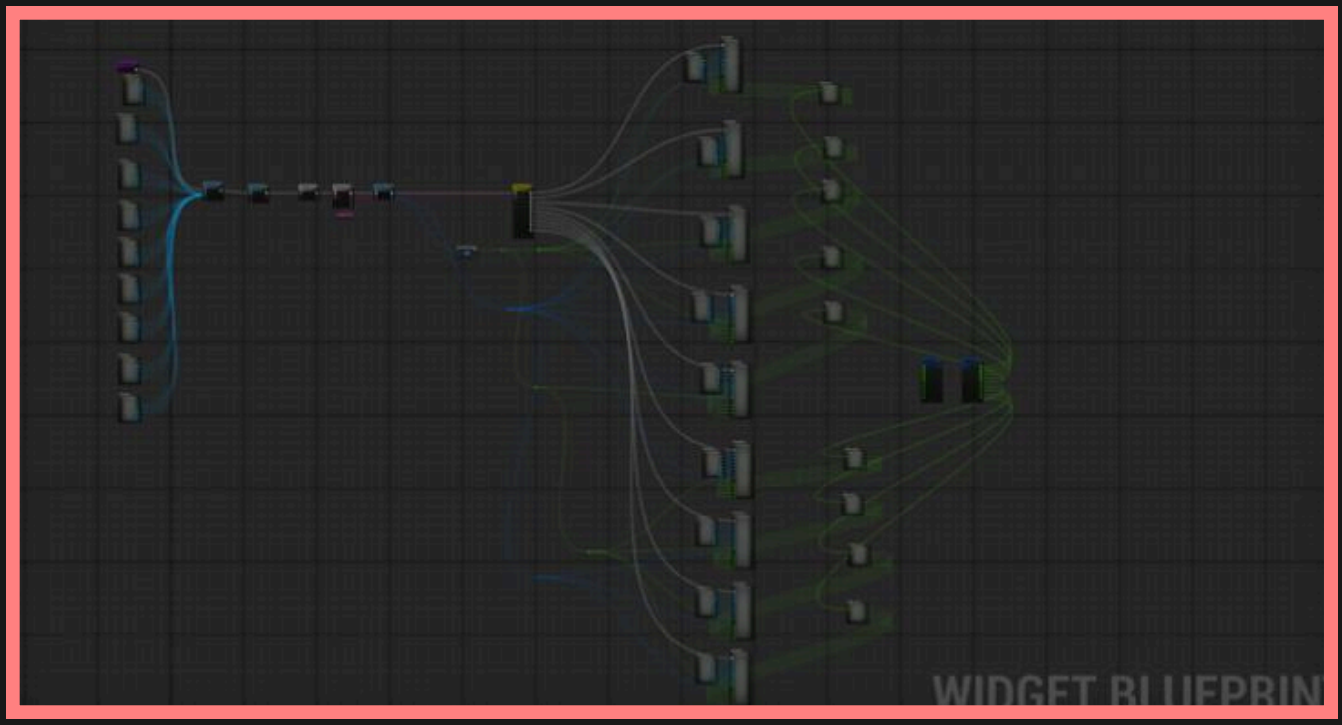
Not on the UI design, since that was dealt with by the artist, but making it work and especially the medal and input mapping systems.

Let's talk about systems, systems were a big part of the game and the process of making it. Systems are a part of the game that serves a purpose, not a mechanic (although it might contain them), but something you interact with to make the whole of the game a more complete experience.

Two systems gave me a lot of problems, the medals system and the input remapping system.

Both systems where mostly implemented in blueprints, as dealing with the UI from them is much more direct. Still, a lot of creativity had to be used to be able to make the system work well, allowing players to have progression and a clear objective in the game.

Here is a little preview:



The input mapping took over 10 hours to complete. One reason was that I was actively fighting Unreal, as it did not allow the remapping that I needed, which meant that even more creativity was needed and I had to wrestle with the engine to overcome its limitations to achieve the end goal.

In both cases, there was a big use of c++ with custom save and load functions to allow persistent medals and key mapping between sessions.

Conclusion

The technical process of Botblast was a very very good learning process. It is a game that from a technical perspective has many different facets, that would usually be headed by different developers, or even different groups. Getting to work on all of them, alone, was an incredible learning experience that allowed me to understand and practice many different areas of game programming.

Unreal was a surprise, as stated previously, in the beginning it was a mess, many problems appeared, it was difficult to even manage to open it the first time. But with time, it became a very potent tool, allowing us to develop the idea we had for BotBlast.

Sound

Unfortunately, the sound was not something that was able to be prioritized in BotBlast. Due to the group being small (was supposed to be 4 people but ended being 3) and the two developers that could oversee the sound production, João and Alex, were also in charge of making the game, being the programmer and the artist.

As such, both the music and the sound effects were not given priority and had to be made late into the development cycle, as a band-aid to finish the game.

Character sounds

A. When creating the character sounds, considering that Tara is a robot, and the whimsical and utopian nature of the story of the game, we wanted the sounds to reflect that, and as such, the sounds ended up being close to a "mickey mousing".

B. The satchel was for much of the development an explosion, a default explosion actor in unreal, but the sounds it made were not at all pleasant. When creating the new sound, that was the main consideration, how do we make a sound that even when playing hundreds of time in a play session, is not unpleasant. We decided to go for a "round" sound, a sound without harsh peaks, whether low or high, it is a sound that wouldn't hurt the players ears with time and wouldn't become an annoyance with time
.
C. The jump sound was the same logic, repeated many times, how do we make it not intrusive and pleasant? The decision was for a sound that gave the idea of jumping and movement but that easily blended with the background music.

Music

The music followed the design idea behind the sounds, it had to not be too intrusive since this is a game that, by design, players will play for long periods in the same level. Ideally, we wished to be able to make more tracks for the music, to change depending on the chamber, on the menu, etc. But as discussed earlier, that plan had to be shelved.

Development Summary

By the end of the semester, the game was fully playable, and the development team managed to deliver on the promised features.

There were a few bumpy roads during development, especially during the start, given Visual Studio's uncooperative behaviour, but we pushed through.

We consider the development of BotBlast a success, and we've learned a lot from it, gaining Game and Level Design experience, programming experience, and 3D Art for Games experience we didn't have before.

Universidade Lusófona do Porto

Bot Blast

Game Design Document

Alex Nogueira
Guilherme Isidoro
João Miguel

Videojogos e Aplicações Multimédia
2024/2025
Projeto de Game Design 2